

04_growth_a

Section 4: Growth Analytics Interview

These answers are deliberately framework-based: every response leans on the canonical growth toolkit - AARRR (Acquisition, Activation, Retention, Referral, Revenue), growth loops, the North Star metric, and prioritization scoring (ICE/RICE) - rather than ad-hoc opinion. The thread tying them together is that growth analytics is about *finding the lever that moves a system*, and Helios is the concrete proof: it diagnoses *why* an e-commerce funnel moved by decomposing the AARRR stages it can observe (acquisition mix and activation funnel) into mix-shift vs rate-change.

Acquisition

Q1. Walk me through how you'd evaluate the channel mix of an e-commerce business. What does a healthy mix look like?

Ideal answer. Channel mix is the share of sessions (and, more importantly, revenue) coming from each acquisition channel, and I evaluate it on three axes at once: volume share, conversion quality, and cost. A healthy mix is *diversified and not fragile* - no single paid channel should carry the majority of revenue, because that creates a single point of failure when CPMs spike or an algorithm changes - and it skews toward channels with a self-reinforcing growth loop (organic search, direct, referral) rather than purely paid pumps. Concretely I'd group every session into the 10 GA4 default channel groups (Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other) and compute `sessions`, `session_conversion_rate`, `revenue`, and `revenue_per_session` per group, then look at the *gap* between volume share and revenue share - a channel that is 30% of sessions but 10% of revenue is a quality problem, not a volume win. The senior move is to never average per-channel conversion rates; you compute mix-level rates as `SUM(purchasing_sessions) / SUM(sessions)` after grouping, otherwise Simpson's paradox makes a healthy-looking blended number hide a deteriorating channel. **Why Helios demonstrates it.** Helios pivots every funnel metric across `channel_group` (one of its 16 governed dimensions) sourced from the single `channel_group_case()` macro, so channel-mix analysis is a first-class query, not a one-off; the Decompose agent then splits any channel-level movement into mix vs rate effects. **Follow-ups.** How would you weight volume vs quality when one channel is high-volume/low-quality? / What blended-mix metric would you put on an executive dashboard? / How does channel mix change the interpretation of a flat overall conversion rate?

Q2. Define CAC. What's the difference between blended CAC and paid CAC, and when does each mislead?

Ideal answer. CAC (customer acquisition cost) is total acquisition spend over a period divided by the number of *new* customers acquired in that period. Blended CAC divides total spend by *all* new customers including the organically acquired ones, while paid CAC divides paid spend only by customers attributable to paid channels - blended always looks better because organic "free" customers dilute the denominator. Blended CAC misleads when you're trying to decide whether to *scale a paid channel*, because the marginal customer from scaling spend costs the paid CAC (often the marginal CAC, which is higher still), not the flattering blend. Paid CAC misleads in the opposite direction - it ignores that paid spend often *seeds* organic and referral loops (brand search, word-of-mouth), so judging a channel purely on its isolated paid CAC undercounts its halo. The mature framing is to report both, watch the CAC:LTV ratio, and treat blended CAC as the board-level efficiency number and channel-level (marginal) CAC as the budget-allocation number.

Why Helios demonstrates it. Helios runs on an observational GA4 dataset with no spend data, so it is honest that it cannot compute true CAC; instead it quantifies *revenue-at-risk in dollars* per channel and sizes experiments, which is the half of the unit-economics equation the data actually supports - exactly the kind of scope honesty that reads as maturity. **Follow-ups.** Why is marginal CAC usually higher than average CAC? / How would payback period change your read on a high-CAC channel? / How do you attribute organic customers that paid spend actually seeded?

Q3. A channel doubled its session volume but overall revenue barely moved. How do you diagnose this?

Ideal answer. This is the canonical channel-quality-vs-volume problem, and the first thing I'd check is whether the new volume converts at all - I'd decompose the change in `revenue` (or `revenue_per_session`) into a mix-shift component and a rate-change component. If a low-converting channel (say Display or a broad Paid Social campaign) doubled, the *mix* effect alone can move blended `session_conversion_rate` down even though every individual channel's conversion rate is unchanged - that's pure composition. The diagnostic SQL is to pull volume share and per-channel conversion for both periods:

```
SELECT channel_group,
       SUM(sessions) AS sessions,
       SUM(purchasing_sessions) / SUM(sessions) AS conv_rate,
       SUM(revenue) / SUM(sessions) AS rev_per_session
FROM helios.marts.fct_daily_funnel
WHERE day BETWEEN @t0_start AND @t1_end
GROUP BY channel_group, period
```

If the doubled channel's `rev_per_session` is far below the others, the volume is real but low-quality and you've simply bought traffic that doesn't buy. The trap to avoid is concluding

"conversion is broken" (a rate story) when it's actually a mix story - those have opposite fixes.

Why Helios demonstrates it. This is the exact scenario Helios was built to disambiguate: `stats-mcp.decompose_change` computes $\text{mix} = \sum \Delta w_i \cdot r_i(t_0)$, $\text{rate} = \sum w_i(t_0) \cdot \Delta r_i$, and `interaction`, so a "volume doubled, revenue flat" case is resolved deterministically into "mix effect dominates, rates held" rather than a hand-waved guess. **Follow-ups.** What would the interaction term tell you here? / How would you size the revenue you'd recover by reallocating spend away from the low-quality channel? / When is buying low-converting volume actually rational (e.g., upper-funnel)?

Q4. Explain first-touch, last-touch, and last-non-direct attribution. Which would you default to and why?

Ideal answer. Attribution assigns credit for a conversion to the touchpoints in the journey: first-touch gives 100% credit to the channel that first acquired the user, last-touch to the final channel before conversion, and last-non-direct (GA4's default) is last-touch but skips direct visits, reassigning credit to the last *campaignable* channel on the theory that "direct" is usually a return visit, not a discovery. I'd default to last-non-direct for day-to-day reporting because it's GA4's native model, it's reproducible, and it avoids over-crediting Direct (which is a junk-drawer bucket - bookmarks, dark social, untagged links). But I'd be explicit that no single-touch model is "correct"; first-touch over-credits discovery channels and starves closers, last-touch does the reverse, and both are wrong for multi-session journeys. For budget decisions I'd want data-driven or position-based multi-touch attribution, and ideally incrementality testing (geo holdouts), because attribution answers "who to credit" while only experiments answer "what was caused." **Why Helios demonstrates it.** Helios reports against GA4's channel-group conventions and is explicit about Direct being a residual bucket; because the dataset is observational, Helios *designs and sizes* quasi-experimental readbacks (pre/post, difference-in-differences) rather than claiming a single attribution model is causal truth. **Follow-ups.** Why does Direct inflate under last-touch, and what does that hide? / When does first-touch actually beat last-non-direct? / How would you validate an attribution model against an incrementality test?

Q5. There's a well-known GA4 traffic_source gotcha that trips up acquisition analysis. What is it and how do you handle it?

Ideal answer. The gotcha is that the event-level `traffic_source` record in the GA4 BigQuery export is the user's *first-touch* source/medium/campaign - it's stamped once at user acquisition and repeats on every event for that user forever - so if you naively `GROUP BY traffic_source.medium`, you are not analyzing *session* acquisition, you're analyzing *user* acquisition, and a user first acquired via Paid Search will have every later organic or direct session mislabeled as Paid Search. This systematically over-credits whatever channel first acquired your users and makes returning-visit channels (Direct, Email) look smaller than they are. The fix is to

derive session-scoped source/medium from the `event_params` (the `source`, `medium`, `campaign`, and `gclid` params attached to the session's `session_start` / `page_view` events) and only fall back to the event-level `traffic_source` when the session-scoped params are null. That's the difference between "where did this user originally come from" and "what brought them *this time*," and channel-mix and conversion-by-channel analysis need the latter. **Why Helios demonstrates it.** This is codified as a hard rule in Helios: it derives session-scoped `source` / `medium` from `event_params` and falls back to first-touch `traffic_source` only when null, so its `channel_group` dimension reflects session acquisition, not lifetime first-touch - removing a silent, systematic bias from every acquisition diagnosis. **Follow-ups.** How would first-touch labeling bias a returning-user cohort analysis? / How do you sessionize source when params are missing mid-session? / What does this gotcha do to a last-non-direct attribution built on top of it?

Q6. How do you compare paid vs organic acquisition beyond just cost?

Ideal answer. Cost is only one axis; I compare paid vs organic on quality, durability, and scalability. Quality means conversion rate, AOV, and downstream activation/retention of the cohort each channel brings - paid traffic from a broad prospecting campaign often converts and retains worse than organic-search intent traffic, so a "cheaper" paid customer can be more expensive on an LTV basis. Durability is the growth-loop question: organic search, direct, and referral are *compounding* (content and brand keep paying off, a flywheel), while paid is a *pump* that stops the instant you stop spending - so a business overweight on paid has a structurally fragile growth engine. Scalability cuts the other way: paid scales fast and predictably (turn up budget), organic scales slowly. So the framework is to map each channel onto a 2x2 of unit economics (CAC:LTV) vs durability (loop vs pump), favor compounding channels for the base, and use paid for controllable, incremental growth on top - always validated with incrementality, since organic and direct are partly *caused* by paid. **Why Helios demonstrates it.** Helios segments the funnel and revenue by `channel_group` and computes `revenue_per_session` per channel, giving the quality (not just volume) read for each; its honesty about lacking spend data keeps the paid/organic comparison to what the observational dataset can defend. **Follow-ups.** How do you isolate the incremental value of organic that paid actually drove? / Which channel would you cut first in a downturn and why? / How does the loop-vs-pump distinction change a forecast?

Q7. What is an acquisition cohort and what questions does it answer that a cross-sectional view can't?

Ideal answer. An acquisition cohort groups users by *when* (and often *how*) they were acquired - e.g., "users first seen in week of 2020-12-07 via Paid Search" - and then tracks that fixed group's behavior forward over time. The questions it answers that a cross-sectional snapshot cannot are about *trajectory and quality drift*: is the December cohort activating and converting better or

worse than the November cohort, and is a channel's customer *quality* degrading even while its volume grows? A cross-sectional "conversion rate this month" blends new and old users and hides the fact that, say, a scaled Paid Social channel is bringing progressively worse cohorts. Cohorting also separates acquisition effects from retention effects: a falling overall active-user count can be a *new-cohort acquisition* problem or an *old-cohort retention* problem, and only a cohort table (acquisition period on the rows, age on the columns) tells you which. The frequent mistake is comparing cohorts of different ages as if they were comparable - you must align on cohort *age*, not calendar date. **Why Helios demonstrates it.** Helios's `stats-mcp.cohort_retention` builds these triangles deterministically, and because identity is cookie-grain (`user_pseudo_id`, since `user_id` is almost always null), it is explicit that cohorts are cookie-based approximations - a stated assumption rather than a hidden one. **Follow-ups.** How would you tell an acquisition problem from a retention problem in a declining MAU? / Why must you compare cohorts at equal age? / What does cookie-grain identity do to cohort sizes over time?

Q8. Define a growth loop and contrast it with a funnel. Give an acquisition-loop example.

Ideal answer. A funnel is a *linear, one-way* model - traffic enters at the top and a shrinking fraction reaches the bottom - whereas a growth loop is *circular*: the output of the loop feeds back as input to the next turn, so growth compounds instead of requiring fresh top-of-funnel spend each time. The classic acquisition loop is content/SEO: a user lands on a page, that engagement and any user-generated content strengthen the page's ranking, which ranks for more queries, which acquires more users - the output (users + content) becomes the input (better ranking). A referral loop is even cleaner: acquired users invite others who become acquired users. The strategic point is that loops have *defensible, compounding* economics while funnels are *leaky and need constant refilling*, so a great growth team instruments the loop's cycle time and amplification factor (how many new users each user generates), not just the funnel's step rates. Funnels and loops are complementary - you still optimize the funnel *inside* each turn of the loop. **Why Helios demonstrates it.** Helios models the GA4 macro funnel (session_start -> view_item -> add_to_cart -> begin_checkout -> add_shipping_info -> add_payment_info -> purchase) as its diagnostic spine while framing channel mix in loop terms - it can flag when a compounding channel (Organic Search) is losing share to a pump channel, which is a loop-health signal a pure funnel view misses. **Follow-ups.** What two metrics define a loop's health? / How would you instrument referral-loop amplification in GA4? / When is a funnel the right model and a loop the wrong one?

Q9. Acquisition volume is up but the overall funnel conversion rate dropped. Is the funnel broken? How do you prove it isn't (or is)?

Ideal answer. Not necessarily - this is the textbook mix-shift-masquerading-as-rate-change situation, and you prove which it is with a decomposition, not an eyeball. If acquisition grew by

adding sessions from a low-converting channel or device, the *blended*

`session_conversion_rate` falls purely because the weights `w_i` shifted toward low-`r_i` segments, even if every segment's own conversion rate is flat or improving (Simpson's paradox). To prove it, I decompose $\Delta(\text{conversion_rate})$ into $\text{mix} = \sum \Delta w_i \cdot r_i(t_0)$ (composition moved), $\text{rate} = \sum w_i(t_0) \cdot \Delta r_i$ (behavior moved), and *interaction*. If the mix term explains nearly all of the drop and per-segment rates are stable, the funnel is *not* broken - you just changed who's walking into it. If the rate term dominates, then a real behavioral/UX problem exists and I'd drill the worst-affected segment. The discipline is to *always drill rate effects before mix effects*, because rate effects are real behavior change you can fix, while mix effects are composition artifacts you manage at the acquisition layer. **Why Helios demonstrates it.** This is Helios's reason to exist: the Decompose agent calls `stats-mcp.decompose_change` to split the movement, and the Diagnose agent prioritizes rate effects over mix effects in its hypothesis tree - so "is the funnel broken?" gets a deterministic, dollar-quantified answer instead of a debate.

Follow-ups. What does a large interaction term imply about the two periods? / If mix dominates, what's the recommended action vs if rate dominates? / How do you guard the decomposition against a tiny-sample segment dominating the math?

Q10. How do you detect and quantify a mix-shift in acquisition channels specifically?

Ideal answer. A mix-shift in acquisition is a change in the *volume share* `w_i` of channels between two periods, independent of how each channel converts. I detect it by computing each `channel_group`'s session share in `t0` and `t1`, flagging channels whose share moved beyond a noise threshold, and then quantifying the *business impact* of that shift by holding each channel's own conversion/`revenue_per_session` fixed and asking: how much did blended revenue move purely because the weights changed? That isolated number is the mix effect, $\sum \Delta w_i \cdot r_i(t_0)$. Practically:

```
SELECT channel_group, period,
       SUM(sessions) / SUM(SUM(sessions)) OVER (PARTITION BY period) AS volume_share,
       SUM(revenue) / SUM(sessions) AS rev_per_session
FROM helios.marts.fct_daily_funnel
GROUP BY channel_group, period
```

Then the mix effect is the share deltas weighted by baseline `rev_per_session`. The key sophistication is reporting the mix-shift *in dollars*, not just "Display share rose 6 points," because a share shift toward a high-RPS channel is good news and toward a low-RPS channel is a quantifiable revenue drag - same direction of share change, opposite business meaning. **Why Helios demonstrates it.** Helios's offline benchmark literally injects *volume perturbations* (change `w_i` only, holding rates fixed) as labeled mix-shift scenarios, and grades whether the pipeline rediscovers them; `stats-mcp.decompose_change` returns the mix effect in the metric's units so it can be priced as revenue-at-risk. **Follow-ups.** How big a share change is "signal" vs seasonal

noise? / Why can a mix-shift toward a high-RPS channel still be a risk? / How would you separate a genuine acquisition mix-shift from a tracking/tagging change?

Q11. Where does acquisition sit in AARRR, and what's the single metric you'd watch for acquisition health?

Ideal answer. Acquisition is the first A in AARRR - the top of the pirate-metrics funnel - and it measures whether you can reliably bring qualified strangers to the product. The single metric depends on the business model, but for an e-commerce store I'd watch *qualified new sessions by channel* (or new-user sessions) as the volume pulse, paired with a quality guardrail like `session_conversion_rate` or `revenue_per_session` of the *newly acquired* traffic - because volume without quality is vanity. If forced to one composite, I'd lean toward CAC-payback or new-customer revenue contribution, since it bakes cost and quality together. The framework discipline is that acquisition should never be optimized in isolation: a North Star metric (e.g., weekly purchasing users, or revenue per active user) sits above AARRR and forces acquisition to be judged by the *quality* of users it feeds into Activation and Retention, not raw traffic. Optimizing acquisition volume against a North Star prevents the classic failure of celebrating a traffic spike that never converts. **Why Helios demonstrates it.** Helios treats the acquisition layer as the *source of weights* `w_i` feeding the funnel and always pairs a volume read with `revenue_per_session` per `channel_group`, so acquisition is judged by downstream quality - the AARRR-into-North-Star linkage made concrete. **Follow-ups.** Why is raw session volume a vanity metric? / How would you design a North Star that keeps acquisition honest? / What guardrail metric stops acquisition from gaming the North Star?

Q12. How would you prioritize which acquisition channel to invest in next? Use a scoring framework.

Ideal answer. I'd use a RICE or ICE scoring pass over candidate channel bets so prioritization is transparent and arguable on inputs, not vibes. ICE scores each option on Impact (expected revenue lift if it works), Confidence (how sure I am, grounded in cohort quality and prior data), and Ease (effort/cost to test), and I multiply or average them; RICE adds Reach (how many users the bet touches) and divides by Effort: $RICE = (Reach \times Impact \times Confidence) / Effort$. For acquisition specifically, Impact should be the *dollar* opportunity (e.g., the revenue-at-risk or revenue-upside a decomposition surfaced for that channel), Confidence should be pulled down for channels where the data is observational or the cohort is small, and Effort includes both creative/ops cost and the experiment runtime needed to read a result. The senior nuance is that the highest raw-ICE channel isn't always the pick - you weight toward *compounding* (loop) channels for long-run defensibility and reserve fast paid bets for filling near-term gaps. The output is a ranked, justifiable backlog, not a single hero bet. **Why Helios demonstrates it.** Helios's Prescribe agent emits a *prioritized, powered* experiment backlog - each card carries the dollar impact (from the decomposition), a confidence grounded in significance testing, and a

runtime estimate from `experiment-mcp ( power_analysis / runtime_estimate )` - which is exactly the Impact/Confidence/Effort triad RICE formalizes. **Follow-ups.** When does the top-RICE item not get built? / How do you keep Confidence honest on observational data? / How does experiment runtime feed the Effort term?

Q13. How do seasonality and one-off campaigns confound acquisition trend analysis, and how do you control for them?

Ideal answer. Acquisition traffic is highly seasonal and event-driven - Black Friday, Cyber Monday, holiday gifting, a product launch, or a one-day email blast can swing channel volumes and mix dramatically - so a naive period-over-period comparison will mislabel a calendar effect as a real change. I control for this three ways: compare against the right baseline (year-over-year or the same weekday/week-of-season, not just last week), maintain a *known-events calendar* so a spike on a launch date isn't flagged as an anomaly, and use a forecast with prediction intervals (e.g., a seasonal model) so I'm comparing actuals against an expected band rather than a flat line. The decomposition then runs on the *residual* - the part of the movement the seasonal expectation didn't already explain. The failure mode to avoid is "seasonality theater," where every real regression gets dismissed as "probably just seasonal"; the discipline is to quantify the seasonal expectation explicitly and require the residual to clear a significance bar before raising an alarm.

Why Helios demonstrates it. Helios's `stats-mcp.forecast` (Prophet/statsmodels) sets the expected band, its memory store holds a seasonality/launch calendar, and the Critic agent runs an explicit *seasonality* refutation that can DOWNGRADE or DROP a finding - and the offline benchmark includes deliberate "seasonality decoy" scenarios to ensure Helios doesn't fire on calendar effects. **Follow-ups.** What baseline do you pick when there's no prior-year data? / How does a launch-calendar entry change anomaly scoring? / How do you avoid over-suppressing real regressions as "seasonal"?

Activation

Q14. Define activation. How is it different from acquisition and from conversion?

Ideal answer. Activation is the second A in AARRR: the moment a newly acquired user first experiences the product's core value - the point at which they "get it" and become meaningfully more likely to stick. It's distinct from acquisition (which only got them in the door - a `session_start`) and from a one-time conversion (which is a transaction, a Revenue-stage event); activation is specifically about *first realized value*, which for a SaaS product might be "completed setup and ran a first report" and for e-commerce is closer to "found a relevant product and engaged with it" (a `view_item` or `add_to_cart`), with purchase being the eventual conversion downstream. The reason the distinction matters operationally is that the *fix* differs: low acquisition is a marketing problem, low activation is an onboarding/first-experience problem, and low

conversion can be a pricing/checkout problem. Conflating them sends you optimizing the wrong stage. A precise activation definition is the prerequisite for an activation *rate*, which is the lever between traffic and retention. **Why Helios demonstrates it.** Helios's funnel separates the acquisition entry (`session_start`), the activation/engagement steps (`view_item` , `add_to_cart`), and the conversion steps (`begin_checkout` -> `purchase`) as distinct, separately-decomposable stages - so a drop can be localized to activation rather than blamed on acquisition or checkout. **Follow-ups.** What's the e-commerce equivalent of a SaaS "aha moment"? / Why does conflating activation and conversion mislead the fix? / Where does `engagement_rate` fit relative to activation?

Q15. What is the "aha moment" and how would you find it empirically?

Ideal answer. The aha moment is the specific early action (or set of actions) that correlates most strongly with a user becoming a retained, valuable customer - Facebook's "7 friends in 10 days" and Slack's "2,000 messages" are the canonical examples. I find it empirically by taking an acquisition cohort, enumerating candidate early behaviors (e.g., viewed N items, added to cart, used search, returned within X days), and measuring which behavior best *separates* retained/converted users from churned ones - i.e., the action where the gap in downstream retention between users who did it and didn't is largest and the curve flattens (a threshold effect). I'm careful to frame this as *correlational* and then validate causality: the right test is whether *driving* more users to that action via onboarding actually lifts retention (a quasi-experiment or A/B), because otherwise I might just be describing what engaged users do anyway, not what *causes* engagement. The output is a crisp, measurable activation definition the whole team can rally around. **Why Helios demonstrates it.** Helios is explicit that its dataset is observational, so it surfaces *candidate* leading indicators by decomposing the activation funnel and then has the Prescribe agent *design and size the experiment* to test whether driving that behavior is causal - it never claims correlation is causation, which is the exact discipline aha-moment analysis requires. **Follow-ups.** How do you avoid mistaking a correlate of engagement for a cause? / What sample size do you need to trust the threshold? / How would you A/B test driving users to the aha action?

Q16. Define activation rate precisely and write how you'd compute it.

Ideal answer. Activation rate is the share of newly acquired users (or sessions) who reach the activation milestone within a defined window - precision requires nailing the numerator (the activation event), the denominator (which users, over what acquisition window), and the time bound (within N days of acquisition). For the GA4 store, a defensible session-grain definition is "sessions that reached the engagement/value step out of all sessions," which maps to a funnel-step rate:

```

SELECT
  channel_group,
  SUM(view_item_sessions) AS activated_sessions, -- reached value step
  SUM(sessions) AS sessions,
  SUM(view_item_sessions) / SUM(sessions) AS activation_rate
FROM helios.marts.fct_daily_funnel
GROUP BY channel_group

```

Two non-negotiables: compute the rate as `SUM(numerator)/SUM(denominator)` after grouping (never the average of per-segment ratios - that reintroduces Simpson's paradox), and use the monotonic "reached" definition so the numerator counts a session that reached the value step *or any later step*, keeping the rate bounded by 1. The exact milestone is a product choice; the rigor is in fixing the cohort and window so the rate is comparable over time. **Why Helios demonstrates it.** Helios's `int_ga4__funnel_steps` uses `reached_*` max-downstream monotonic flags (guaranteeing `sessions ≥ reached_view_item ≥ ... ≥ reached_purchase`, so every step rate is `≤ 1`), and the semantic layer computes rates as `SUM(num)/SUM(den)` - so an activation-rate metric is governed and Simpson-safe by construction. **Follow-ups.** Why use a monotonic "reached" flag instead of a per-step "did" flag? / How does the activation window length change the rate? / Session-grain vs user-grain activation - which and why here?

Q17. Walk me through how you'd map and diagnose an onboarding (activation) funnel.

Ideal answer. I map the onboarding funnel as the ordered sequence of steps from entry to first value, measure the *step-to-step* conversion at each, and look for the biggest drop-off step relative to a benchmark - that's where the leak is. For the store the activation segment of the funnel is `session_start → view_item → add_to_cart`, and I'd compute `view_to_cart_rate` and the step rates around it, then segment the worst step by device, channel, and landing page to localize the problem (e.g., the drop is concentrated on mobile, or on traffic from one channel landing on a slow page). Diagnosing *why* a step fell requires the mix-vs-rate split again: did the step rate genuinely drop (a real UX/onboarding problem) or did the *composition* of users hitting that step shift toward a segment that always converted worse? Only after isolating a real rate drop in a real segment do I form a hypothesis (e.g., a regression on the mobile product page) and prescribe a fix. The framework is: ordered steps -> biggest leak -> segment the leak -> decompose mix vs rate -> hypothesis -> sized experiment. **Why Helios demonstrates it.** Helios's Decompose agent runs on each funnel step (`view_to_cart_rate`, `cart_to_checkout_rate`, `checkout_to_purchase_rate`) and the Diagnose agent runs a best-first hypothesis-tree RCA that's SQL-verified at every node - so an onboarding-funnel leak is localized to a step *and* a segment *and* attributed to rate-not-mix, then handed to Prescribe. **Follow-ups.** How do you pick the benchmark for a "good" step rate? / What if the leak is split across two steps? / How do you tell a real step regression from a composition shift at that step?

Q18. What is time-to-value (TTV) and why does it matter for activation?

Ideal answer. Time-to-value is the elapsed time from a user's first contact (acquisition) to the moment they first experience core value (activation) - shorter TTV means users hit the aha moment before they lose interest, so it's one of the strongest levers on activation rate and, downstream, on retention. It matters because attention decays fast: every extra step, form field, or minute before first value bleeds users, and reducing TTV often lifts activation more cheaply than buying more traffic. I'd measure it as the median (not mean, because it's right-skewed) time or step-count from `session_start` to the activation event, segment it (mobile TTV is usually worse), and treat a long TTV at a specific step as a prioritized onboarding fix. For e-commerce, TTV maps to "how fast can a visitor find a relevant product" - search quality, navigation, and page speed - so it ties directly to the early funnel steps. The framework link is that TTV is the *speed* dimension of activation, while activation rate is the *completion* dimension; you want both high-completion and low-time. **Why Helios demonstrates it.** Helios's sessionized data carries event timestamps, so the time and step distance from `session_start` to the value step (`view_item` / `add_to_cart`) is derivable and decomposable by segment - letting a TTV regression be localized (e.g., mobile) the same way a rate regression is. **Follow-ups.** Why median over mean for TTV? / How would you trade off TTV against richness of the first experience? / How does TTV differ on mobile vs desktop and why?

Q19. What are leading indicators of retention, and how do they relate to activation?

Ideal answer. Leading indicators of retention are *early*, observable behaviors that predict whether a user will stick around long before the retention itself is measurable - they're the bridge between activation and the Retention stage of AARRR, and activation milestones are usually the strongest ones. Examples: reaching the aha action within the first session/week, returning within N days of first visit, breadth of early engagement (viewed multiple categories, used search), and for e-commerce, adding to cart or completing a first purchase. They matter because retention is a *lagging* metric - you can't wait 90 days to learn a cohort is bad - so you instrument leading indicators to get a same-week read on cohort quality and to know which onboarding changes are working. The analytical method is the same as finding the aha moment: regress/segment early behaviors against eventual retention, pick the ones with the strongest, most stable separation, and validate causally. The mature caveat is that a leading indicator must be validated as *predictive out-of-sample*, not just fit in-sample, or you'll chase a spurious early signal. **Why Helios demonstrates it.** Helios is honest that its ~3-month window (Nov 2020-Jan 2021) defers true long-horizon retention/LTV, so it reports *short-horizon retention proxies with stated assumptions* and uses `stats-mcp.cohort_retention` to relate early activation behaviors to the near-term return signal - leading indicators done with explicit limitation-flagging. **Follow-ups.** How do you validate a leading indicator is predictive, not just correlated? / What proxy would you use when you only have 90 days of data? / How would activation rate itself serve as a leading indicator?

Q20. Activation rate dropped this week. Walk me through your diagnosis before you alarm anyone.

Ideal answer. First I'd check data quality - a tracking break, a tag misfire, or a schema change can fake an activation drop, and shipping a false alarm burns credibility - so I reconcile the event volumes against canonical totals before trusting the number. Then I'd ask whether the drop clears the noise/seasonal band: compare against the right baseline and a forecast interval, not last week's point. If it's real, I decompose `Δ(activation_rate)` into mix vs rate: a shift in *who* arrived (e.g., a surge of low-intent Display traffic) can drop blended activation with every segment's own rate intact (mix), versus a genuine in-segment drop (rate) that signals an onboarding/UX regression. If rate dominates, I segment to localize it (device, channel, landing page), form a hypothesis, run a significance test to confirm the affected-segment drop isn't sampling noise, and only then quantify the dollar impact and raise it - with a recommended, sized experiment to fix it. The order is deliberate: data-quality -> seasonality -> mix-vs-rate -> segment -> significance -> dollars -> action. **Why Helios demonstrates it.** This is the Helios pipeline end to end: `warehouse-mcp.reconcile` ($\leq 0.5\%$ drift) guards data quality, `stats-mcp.forecast` sets the band, `decompose_change` splits mix/rate, `significance_test` confirms the segment, and the Critic adversarially attacks the finding (data-quality / seasonality / insufficient-sample / mix-confound) before the Narrator writes the Decision Brief. **Follow-ups.** Why is data-quality the first check, not the decomposition? / What significance bar would you require on the affected segment? / How do you price an activation drop in dollars when activation isn't a purchase?

Q21. How would you design an experiment to improve activation, and how do you size it?

Ideal answer. I'd start from a diagnosis (which step leaks, in which segment, mix vs rate) so the experiment targets a real, sized opportunity, then write a hypothesis with one primary metric (the activation rate at the leaking step) and a guardrail (e.g., downstream purchase rate, so I don't lift activation at the cost of conversion). Sizing is a power-analysis problem: given the baseline rate, the minimum detectable effect (MDE) worth shipping, and target power (0.8) and significance (0.05), I compute the required sample per arm and then translate it into runtime given the segment's daily traffic - a low-traffic segment may need weeks, which itself is a prioritization input. I'd guard against the classic mistakes: peeking (fixing the horizon up front or using sequential methods), under-powering (calling a flat result a "no effect" when the test couldn't detect the MDE), and ignoring the guardrail. If a live A/B isn't possible, I design a quasi-experiment - a pre/post or difference-in-differences readback against a comparable unaffected segment as the control. **Why Helios demonstrates it.** Helios's `experiment-mcp` does exactly this - `power_analysis` -> `runtime_estimate` -> `design_experiment` - and because the GA4 data is observational, Prescribe emits quasi-experimental (pre/post, diff-in-diff) readback designs, with each card carrying the dollar impact and runtime that feed the RICE-style backlog ranking.

Follow-ups. What guardrail metric stops you gaming activation? / How does low segment traffic change your design? / Why is peeking at the result dangerous and how do you prevent it?

Q22. How does the mix-shift-vs-rate-change distinction apply inside the activation funnel, not just acquisition?

Ideal answer. It applies at *every* rate in the funnel, including each activation step, because any aggregate step rate is $R = \sum w_i \cdot r_i$ over segments - so a change in `view_to_cart_rate` can be driven by a shift in *who* reached `view_item` (mix) or by a genuine change in how each segment behaves at that step (rate), exactly as at the acquisition layer. The practical consequence is that an "activation is dropping" alarm must be decomposed before it's actioned: if mobile traffic share rose and mobile always carries a lower view-to-cart rate, blended activation falls with no real UX regression (mix), and the lever is upstream acquisition targeting, not the product page. If instead the desktop view-to-cart rate itself fell, that's a real activation regression (rate) and the lever is the onboarding/page experience. Same number, opposite owner and opposite fix - which is precisely why you never act on a blended funnel rate without splitting it. This is also how you dissolve Simpson's paradox *inside* activation, where overall activation can drop while every segment improves. **Why Helios demonstrates it.** Helios's Decompose agent runs `decompose_change` on the funnel-step rates (`view_to_cart_rate` , `cart_to_checkout_rate` , etc.), not just on acquisition volume - so the mix/rate/interaction identity is applied throughout activation, and the offline benchmark's rate vs volume perturbations test both layers. **Follow-ups.** Give a Simpson's-paradox example inside activation. / How do you decide which segmentation dimension to decompose by? / If interaction is large at a step, what does that say?

Q23. What activation metric would you nominate as (or feed into) a North Star, and why?

Ideal answer. A North Star metric should capture *delivered customer value* and predict long-term growth, so for an e-commerce store I'd nominate something like *weekly purchasing users* (or revenue per active user), and the *activation* metric that feeds it is "share of new users who reach first value within their first session/week." The logic is the AARRR chain: activation is the leading lever that converts acquired traffic into the retained, revenue-generating users the North Star counts, so an activation milestone is a high-quality *input* metric to the North Star rather than the North Star itself. I'd avoid making raw activation events the North Star, because they can be gamed (lower the value bar and "activation" rises while real value doesn't) - which is why the North Star sits above AARRR and forces every stage, including activation, to be judged by downstream value. The test of a good North Star is that moving it reliably moves revenue and retention; activation rate earns its place because it demonstrably leads both. **Why Helios demonstrates it.** Helios's metric registry (28 metrics, including `revenue_per_user` /ARPU and `revenue_per_session`) lets activation milestones be evaluated against downstream value metrics in the same governed query - so the activation-to-North-Star linkage is computed on one

source of truth, not stitched across tools. **Follow-ups.** Why not make the activation event itself the North Star? / How would you detect if activation is being gamed? / What guardrails sit beside a purchasing-users North Star?

Q24. How do you connect activation back to acquisition channel quality?

Ideal answer. Activation is the cleanest *quality* signal for an acquisition channel, because it measures whether the users a channel brings actually engage with core value, not just whether they showed up - so I cross-tabulate activation rate by `channel_group` (and campaign) on the *new-user* cohort and compare. A channel with high session volume but low activation rate is bringing low-intent traffic, and that's a budget-reallocation signal even before you get to purchase - it tells you the problem is upstream targeting, not the product. This closes the AARRR loop between the two stages: you judge acquisition not by volume but by the activation (and eventual retention) of the cohorts it delivers, which is exactly how you avoid scaling a channel that pumps in traffic that never activates. The analytical care is to compare channels at equal cohort age and to decompose so you don't confuse a channel's intrinsic quality with a temporary mix-shift in *which* of its sub-segments arrived. **Why Helios demonstrates it.** Because Helios pivots both the activation funnel steps and `revenue_per_session` by `channel_group`, it can directly attribute a low activation rate to a specific channel and price the reallocation opportunity in dollars - making "this channel's traffic doesn't activate" a quantified finding rather than an assertion. **Follow-ups.** What does a high-volume/low-activation channel tell you to do? / How do you separate channel quality from a within-channel mix-shift? / Why compare activation at equal cohort age across channels?

Q25. Tie it together: how do AARRR, growth loops, and the mix-vs-rate lens combine into one operating model for diagnosing growth?

Ideal answer. AARRR gives the *map* - five stages (Acquisition, Activation, Retention, Referral, Revenue) where growth can be won or lost - and the first diagnostic move is always to localize a movement to a stage so you fix the right thing. Growth loops give the *engine* view layered on top: which stages feed back into each other (referral and content loops turn Revenue/Activation back into Acquisition), so you optimize for compounding, not just plugging leaks in a linear funnel. The mix-vs-rate lens is the *microscope* applied at every stage and every rate: any aggregate metric is $\sum w_i \cdot r_i$, so before acting you decompose each movement into composition change (mix), behavior change (rate), and interaction - because they have different owners and opposite fixes, and conflating them (Simpson's paradox) is the single most common analytics error. Put together: AARRR tells you *where*, the decomposition tells you *why* (mix or rate), significance tells you *whether it's real*, dollarization tells you *how much it matters*, and a powered experiment tells you *what to do* - and the loop view keeps you investing in compounding stages. That sequence is a defensible operating model, not a dashboard. **Why Helios demonstrates it.** Helios is this

operating model instantiated end to end: Monitor localizes the movement (AARRR stage), Decompose runs the mix/rate identity, the Critic refutes it, Prescribe sizes the experiment, and the Narrator ships a dollar-quantified Decision Brief - grounded in governed SQL (semantic-mcp), deterministic math (stats-mcp), and graded by an 85%-vs-45% offline benchmark. **Follow-ups.** Where would you start a diagnosis if you could only look at one AARRR stage? / How does the loop view change which leak you fix first? / What makes this an operating model rather than just reporting?